Energy Optimization Platform using Machine Learning and Heuristic Models

2025

Contents

Introduction **2**

1 Mathematical Model **3**

Mathematical Model **3**

2 Conceptual Approach to Warm-Start with Machine Learning **11**
2.1 Motivation and Objectives 11
2.2 Operating Principle 11
2.2.1 Features and Interpretation 11
2.3 Ranking Target 12
2.4 Conceptual Pipeline 12
2.5 Expected Benefits 13
2.6 Final Considerations 13

3 Implementation of the Models and Development of Heuristics **14**
3.1 Context and Purpose 14
3.2 Main Data Structures 15
3.3 Class `Scheduler` 15
3.4 Feasibility Check (`_feasible`) 16
3.5 Objective Function 17
3.6 Public Method `run` 17

4 Experimental Results **18**
4.1 Experimental Setup 18
4.2 Summary of Results 18
4.3 Gap Analysis with Respect to the Optimum 19
4.4 Impact of Local Search 19
4.5 Results Analysis 20
4.6 Final Considerations 21

Bibliography **22**

Introduction

This project addresses the problem of operational planning for a small or medium-sized enterprise (SME), taking into account constraints on human resources, time scheduling, and the proper utilization of machinery. The main objective is to propose mathematical models capable of optimizing the use of available resources and reducing operational costs, with particular attention to the peak power demand.

In this PDF, we will present only the aspects related to the operations research component, excluding the full software tool development.

Chapter 1

Mathematical Model

Premise

We introduce the resource "employees", available in a limited number for each 5-minute slot. Each activity requires a certain number of employees throughout its entire duration. This implementation choice slightly complicated our coding phase but made our model much more realistic and applicable to real-world scenarios.

Notation common to both models

- $I = \{1, 2, \ldots, n\}$: set of activities.
- $M = \{1, 2, \ldots, m\}$: set of machines.
- $\mathcal{P} \subseteq I \times I$: pairs (i, j) with precedence constraint $i \to j$.
- $\mathcal{C} = \{(i, k) \in I \times I : i < k \text{ and } \exists\, p \in M \text{ s.t. } R_{i,p} = R_{k,p} = 1\}$: pairs of activities sharing at least one machine.
- $\Delta = \frac{5 \text{ min}}{60 \text{ min/h}} = \frac{1}{12}$ h.
- T: total working day duration in hours.
- $H = 12\,T$: number of 5-minute slots.
- $t_\ell = \ell\,\Delta$, for $\ell = 0, 1, \ldots, H$.
- d_i: duration of activity i (in hours); for simplicity, we assume d_i is a multiple of Δ.

- $D_i = d_i/\Delta \in \mathbb{Z}_{>0}$: duration of i in number of 5-minute slots.
- $[a_i, b_i]$ (hours): *time window* for i, within which i must be **fully** executed.
- $R_{i,p} \in \{0, 1\}$: 1 if activity i requires machine p, 0 otherwise.
- $p_p > 0$: hourly power (kW) of machine p.
- $P_i = \sum_{p=1}^{m} R_{i,p}\, p_p$: total power (kW) consumed by i when active.
- $e_i \in \mathbb{Z}_{>0}$: number of employees required for the execution of activity i (constant).
- $E_\ell \in \mathbb{Z}_{\geq 0}$, for $\ell = 0, \ldots, H-1$: number of employees available in slot ℓ (slot corresponding to $[t_\ell, t_{\ell+1})$).
- M_{big}: sufficiently large "Big-M" constant, e.g. $M_{\text{big}} = H$.

Variables common to both models

$$u_i \in \{0, 1, 2, \ldots, H\}, \quad i = 1, \ldots, n, \quad \text{with } s_i = u_i\, \Delta,$$
$$y_{i,k} \in \{0, 1\}, \quad \forall\, (i,k) \in \mathcal{C},\ i < k,$$
$$z_{i,\ell} \in \{0, 1\}, \quad i = 1, \ldots, n,\ \ell = 0, 1, \ldots, H-1.$$

- u_i: starting slot of i; hence $s_i = u_i\, \Delta$.
- $y_{i,k}$: if $= 1$, i precedes k (prevents overlap on machines); if $= 0$, k precedes i.
- $z_{i,\ell} = 1$ if and only if activity i is running during slot ℓ, that is, $t_\ell \in [\, s_i,\ s_i + d_i\,)$.

Common constraints (for $i = 1, \ldots, n$, $(i,k) \in \mathcal{C}$, $(i,j) \in \mathcal{P}$, $\ell = 0, \ldots, H-1$)

(1) **Time window**:

$$\frac{a_i}{\Delta} \;\le\; u_i \;\le\; \frac{b_i - d_i}{\Delta}.$$

Since a_i, b_i, d_i are multiples of Δ, the bounds are integers.

(2) **Precedence**, for every $(i, j) \in \mathcal{P}$:

$$s_j \;\ge\; s_i + d_i \iff u_j \;\ge\; u_i + D_i.$$

(3) **Non-overlapping on machines** (disjunctive), for every $(i, k) \in \mathcal{C}$, $i < k$:

$$\begin{cases} u_i + D_i \;\le\; u_k + M_{\text{big}}\,(1 - y_{i,k}), \\ u_k + D_k \;\le\; u_i + M_{\text{big}}\,y_{i,k}, \\ y_{i,k} \in \{0, 1\}. \end{cases}$$

(4) **Definition of** $z_{i,\ell}$, for every i, ℓ:

$$\begin{cases} z_{i,\ell} \;\le\; (u_i + D_i - \ell), \\ z_{i,\ell} \;\le\; (\ell + 1 - u_i), \\ z_{i,\ell} \;\ge\; (\ell + 1 - (u_i + D_i)), \\ z_{i,\ell} \;\ge\; (u_i - \ell), \\ z_{i,\ell} \in \{0, 1\}. \end{cases}$$

These constraints enforce $z_{i,\ell} = 1 \iff u_i \le \ell \le u_i + D_i - 1$, i.e., i occupies exactly the slots $u_i, u_i + 1, \ldots, u_i + D_i - 1$.

Explanation:

- $z_{i,\ell} \le u_i + D_i - \ell$: turns $z_{i,\ell}$ off after the activity ends.
- $z_{i,\ell} \le \ell + 1 - u_i$: turns $z_{i,\ell}$ off before it starts.
- $z_{i,\ell} \ge \ell + 1 - (u_i + D_i)$: turns $z_{i,\ell}$ on until the last slot.
- $z_{i,\ell} \ge u_i - \ell$: turns $z_{i,\ell}$ on from slot u_i onwards.

- Together, these imply $z_{i,\ell} = 1$ iff $\ell \in \{u_i, u_i + 1, \ldots, u_i + D_i - 1\}$.

(5) **Human resource (employees) constraint**, for each slot $\ell = 0, \ldots, H-1$:

$$\sum_{i=1}^{n} e_i \; z_{i,\ell} \;\leq\; E_\ell.$$

In each slot ℓ (duration Δ), the total number of employees required by all active activities cannot exceed the available E_ℓ.

Common variable domains:

$$u_i \in \{0, \ldots, H\}, \quad y_{i,k} \in \{0, 1\}, \quad z_{i,\ell} \in \{0, 1\}, \quad \forall i, k, \ell.$$

Model 1: Peak Power Minimization

All constraints (1)–(4) and (Human Resource Constraint) are from the common section. We add the following:

Additional variable for Model 1

$$v \in \mathbb{R}_{\geq 0}: \quad \text{maximum instantaneous power peak.}$$

Power Peak Constraint, for each $\ell = 0, 1, \ldots, H$:

$$\sum_{i=1}^{n} P_i \; z_{i,\ell} \;\leq\; v.$$

Here $P_i = \sum_{p=1}^{m} R_{i,p}\, p_p$ is the hourly power (kW) consumed by activity i when active. By imposing $\sum_i P_i \, z_{i,\ell} \leq v$ for all ℓ, we ensure that

$$v \;\geq\; \max_{\ell=0,\ldots,H} \Big\{ \sum_{i=1}^{n} P_i \, z_{i,\ell} \Big\} \;=\; \max_{t \in [0,T]} C(t).$$

Objective Function – Model 1:

$$\boxed{\min\ v.}$$

Additional Domains – Model 1:

$$v \geq 0.$$

Complete Model 1 with Employee Resource

$$\min\ v$$

$$\text{s.t. } \frac{a_i}{\Delta} \ \leq\ u_i \ \leq\ \frac{b_i - d_i}{\Delta}, \quad \forall\, i = 1, \ldots, n, \tag{1}$$

$$u_j \ \geq\ u_i + D_i, \quad \forall\, (i,j) \in \mathcal{P}, \tag{2}$$

$$\begin{cases} u_i + D_i \ \leq\ u_k + M_{\text{big}}(1 - y_{i,k}), \\ u_k + D_k \ \leq\ u_i + M_{\text{big}}\, y_{i,k}, \qquad\qquad \forall\, (i,k) \in \mathcal{C},\ i < k, \\ y_{i,k} \in \{0,1\}, \end{cases} \tag{3}$$

$$\begin{cases} z_{i,\ell} \ \leq\ u_i + D_i - \ell, \\ z_{i,\ell} \ \leq\ \ell + 1 - u_i, \\ z_{i,\ell} \ \geq\ \ell + 1 - (u_i + D_i), \qquad i = 1, \ldots, n,\ \ell = 0, \ldots, H-1, \\ z_{i,\ell} \ \geq\ u_i - \ell, \\ z_{i,\ell} \in \{0,1\}, \end{cases} \tag{4}$$

$$\sum_{i=1}^{n} e_i\ z_{i,\ell} \ \leq\ E_\ell, \quad \ell = 0, \ldots, H-1, \tag{5}$$

$$\sum_{i=1}^{n} P_i\ z_{i,\ell} \ \leq\ v, \quad \ell = 0, \ldots, H, \tag{6}$$

$$u_i \in \{0, \ldots, H\}, \quad y_{i,k} \in \{0,1\}, \quad z_{i,\ell} \in \{0,1\}, \quad v \geq 0. \tag{7}$$

Model 2: Time-of-Use Cost Minimization

All constraints (1)–(4) and the (Human Resource Constraint) are the same as in the common section. Model 2 does not introduce variable v nor constraints (6)–(7) from Model 1; instead, it includes only the cost-based objective function.

Objective Function – Model 2:

$$\boxed{\min \sum_{\ell=0}^{H-1} \Big(\sum_{i=1}^{n} P_i \, z_{i,\ell} \Big) \;\times\; \Delta \;\times\; c_\ell .}$$

- For each slot ℓ, the total instantaneous power is $C(t_\ell) = \sum_{i=1}^{n} P_i \, z_{i,\ell}$.
- The energy consumed in $[t_\ell,\, t_{\ell+1})$ is $C(t_\ell)\,\Delta$ [kWh].
- The unit cost for that slot is c_ℓ [€/kWh].
- Hence, the cost for the slot is $C(t_\ell)\,\Delta\, c_\ell$.
- Summing over all slots $\ell = 0, \ldots, H-1$ gives the total energy cost.

Complete Model 2 with Employee Resource

$$\min \sum_{\ell=0}^{H-1} \Big(\sum_{i=1}^{n} P_i \, z_{i,\ell} \Big) \, \Delta \, c_\ell$$

$$\text{s.t.} \quad \frac{a_i}{\Delta} \;\le\; u_i \;\le\; \frac{b_i - d_i}{\Delta}, \quad \forall\, i = 1, \ldots, n, \tag{1}$$

$$u_j \;\ge\; u_i + D_i, \quad \forall\, (i,j) \in \mathcal{P}, \tag{2}$$

$$\begin{cases} u_i + D_i \;\le\; u_k + M_{\text{big}}(1 - y_{i,k}), \\ u_k + D_k \;\le\; u_i + M_{\text{big}}\, y_{i,k}, & \forall\, (i,k) \in \mathcal{C},\ i < k, \\ y_{i,k} \in \{0,1\}, \end{cases} \tag{3}$$

$$\begin{cases} z_{i,\ell} \;\le\; u_i + D_i - \ell, \\ z_{i,\ell} \;\le\; \ell + 1 - u_i, \\ z_{i,\ell} \;\ge\; \ell + 1 - (u_i + D_i), & i = 1, \ldots, n,\ \ell = 0, \ldots, H-1, \\ z_{i,\ell} \;\ge\; u_i - \ell, \\ z_{i,\ell} \in \{0,1\}, \end{cases} \tag{4}$$

$$\sum_{i=1}^{n} e_i \; z_{i,\ell} \;\le\; E_\ell, \quad \ell = 0, \ldots, H-1, \tag{5}$$

$$u_i \in \{0, \ldots, H\}, \quad y_{i,k} \in \{0,1\}, \quad z_{i,\ell} \in \{0,1\}. \tag{6}$$

Explanation of the added "Employee Resource"

- e_i: number of employees constantly required during the entire execution of activity i.
- E_ℓ: number of employees available in slot ℓ (given as input for each ℓ).
- The constraint

$$\sum_{i=1}^{n} e_i \, z_{i,\ell} \;\le\; E_\ell \quad \forall\, \ell = 0, \ldots, H-1$$

ensures that in each 5-minute slot, the total number of employees required by all active activities ($z_{i,\ell} = 1$) does not exceed the availability E_ℓ.

Chapter 2

Conceptual Approach to Warm-Start with Machine Learning

2.1 Motivation and Objectives

The purpose of the Machine Learning-based warm-start is to provide the scheduler with an initial ordering of activities that reduces convergence time and improves the quality of the solution compared to a fully random start. The model learns, from examples of optimal or high-quality schedules, to recognize which characteristics of a job determine its priority in the scheduling sequence.

2.2 Operating Principle

The key idea is to treat each activity as an instance of a ranking problem: the model predicts a "criticality score" (or potential contribution to the objective function) based on descriptive features, and then the scheduler performs a descending sort of these scores to initialize the greedy phase.

2.2.1 Features and Interpretation

The features describe the "context" of each job:

- duration and estimated energy contribution define the intrinsic load;

- time windows and slack indicate scheduling flexibility;
- human resources and required machines reflect competition for capacity;
- average energy price and cost variations during the period guide displacement decisions.

This information allows the model to assess the impact of scheduling a job earlier or later in the plan.

2.3 Ranking Target

Instead of labeling activities into discrete categories, a continuous value (delta) is used to measure the performance difference when a job is forced into a specific position compared to the baseline schedule. This approach enables ordering the activities according to the actual importance of their placement within the schedule.

2.4 Conceptual Pipeline

The model generation and application pipeline consists of three stages:

1. **Reference instance generation**: scheduling examples produced via exact solvers or advanced heuristics, used to compute the delta value of each job.
2. **Ranker training**: learning the relationships between features and delta values, obtaining a model capable of predicting the criticality score.
3. **Warm-start application**: for a new instance, features are extracted, scores are computed, and jobs are sorted accordingly to feed the subsequent greedy algorithm.

2.5 Expected Benefits

This conceptual approach allows for:

- reducing the number of explored moves during simulated annealing;
- improving solution quality already in the initialization phase;
- leveraging historical data or previous simulations to guide decisions;
- maintaining modularity and flexibility in integrating different ML models.

2.6 Final Considerations

The ML-based warm-start is not intended to replace optimization heuristics, but rather to enhance them by providing an intelligent starting point learned from data. In this way, the computational efficiency of greedy heuristics is combined with the predictive capability of ranking models.

Chapter 3

Implementation of the Models and Development of Heuristics

3.1 Context and Purpose

The file `scheduler.py` implements the algorithm that, starting from an input JSON file, produces a daily *schedule* in 5 slots. The problem accounts for:

- **precedence** constraints between activities;
- availability windows (`earliest`–`latest`);
- consumption and conflicts of **machines**;
- a limit on simultaneous **employees**;
- hourly **energy** prices.

Two alternative objectives are supported:

1. minimize the **energy cost**;
2. minimize the **power peak**.

3.2 Main Data Structures

Class `Activity`

`id`	string identifier
`duration`	duration (in 5 slots)
`earliest`, `latest`	bounds of the allowed time window
`power_kw`	constant power in
`employees`	required employees
`machines`	list of machines to be locked

Class `Instance`

Loads the JSON and prepares:

1. time grid $\Delta = 5$ and horizon $H = 288$ slots;
2. slot-wise price vector ($24 \to 288$);
3. employee capacity vector (a *scalar* duplicated if constant);
4. dictionary `machine_power`;
5. vector of `Activity`;
6. predecessor list for each job;
7. objective specification (`energy_cost` / `peak_power`).

3.3 Class `Scheduler`

Internal state (`_init_state`)

- `schedule`: job $\to$ start slot.
- `load_kw`, `load_emp`: per-slot load vectors.
- `load_mach`: boolean matrix (machine $\times$ slot).
- `price_cs`: cumulative price prefix: $CS_t = \sum_{u<t} \text{price}_u \frac{\Delta}{60}$.

Initial Construction

With ML model (`_ml_greedy`)

1. Compute 17 features (duration, slack, power, price statistics, ...).
2. Obtain scores with `model.predict`.
3. Insert jobs in descending score order, each time choosing the start that minimizes the objective increase.

Without model (`_deterministic_greedy`) Generate a topological order that respects precedences and use the same incremental insertion rule.

Local Improvement (`_local_search`)

- Simulated Annealing with $T_0 = 1$ and $\alpha = 0.95$.
- **Shift**: move a job by ± 1 slot.
- **Swap**: exchange two jobs if feasibility is preserved.
- A worsening move with increase Δf is accepted with $e^{-\Delta f/T}$.

3.4 Feasibility Check (`_feasible`)

Returns `True` if the proposed placement:

1. respects the time window and $start + duration \leq H$;
2. does not exceed the employee capacity in any involved slot;
3. does not overlap machines already in use;
4. satisfies $\forall p$ predecessors : schedule$[p]$+$duration_p \leq start$.

3.5 Objective Function

Energy Cost

$$f = \sum_{t=0}^{H-1} \text{load_kw}[t] \ \text{price}[t] \frac{\Delta}{60} \quad []$$

Power Peak

$$f = \max_{t} \text{load_kw}[t] \quad []$$

3.6 Public Method `run`

1. Build the initial solution (ML or greedy).
2. (Optional) apply local search.
3. It can be invoked multiple times with different seeds (`restarts`) and returns the best schedule found.

Parameters `max_iter`, `restarts`, and `seed` allow trading off quality and runtime while keeping experiments reproducible.

Chapter 4

Experimental Results

4.1 Experimental Setup

The experiments were conducted on instances generated with a varying number of jobs and machines:

20–10 20 jobs on 10 machines

30–10 30 jobs on 10 machines

50–20 50 jobs on 20 machines

70–20 70 jobs on 20 machines

For each configuration we executed:

- **baseline scheduling** (deterministic multi-start greedy) with the "cost" objective;
- **scheduling with ML warm-start**;
- **exact optimization** with Gurobi to obtain the optimum for both cost and peak.

4.2 Summary of Results

The main outcomes are summarized in the plots shown in Figures 4.1–4.3.

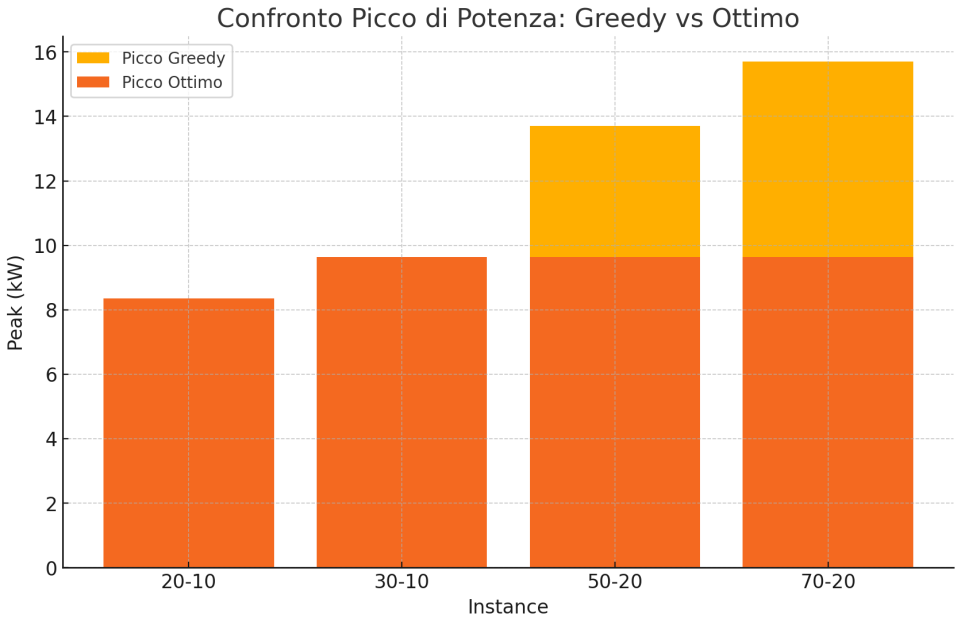


Figure 4.1: Cost comparison: Baseline vs ML vs Optimum.

4.3 Gap Analysis with Respect to the Optimum

The percentage gap is defined as

$$\text{Gap} = \frac{\text{Cost}_{\text{alg}} - \text{Cost}_{\text{opt}}}{\text{Cost}_{\text{opt}}} \times 100.$$

Table 4.1 reports the gaps for the baseline scheduling and for the ML warm-start.

Instance	Baseline Gap (%)	ML Gap (%)	Gap Reduction (p.p.)
20–10	26.37	5.48	20.89
30–10	27.50	7.63	19.87
50–20	27.66	2.86	24.79
70–20	32.03	11.16	20.86

Table 4.1: Percentage gap analysis with respect to the optimal solution.

4.4 Impact of Local Search

To quantify the contribution of the local-search (LS) phase, the four instances were re-run with `improve=False` (greedy+ML construction only) and with `improve=True` (greedy+ML followed by SA). Table 4.2 compares the gaps with respect to the optimum.

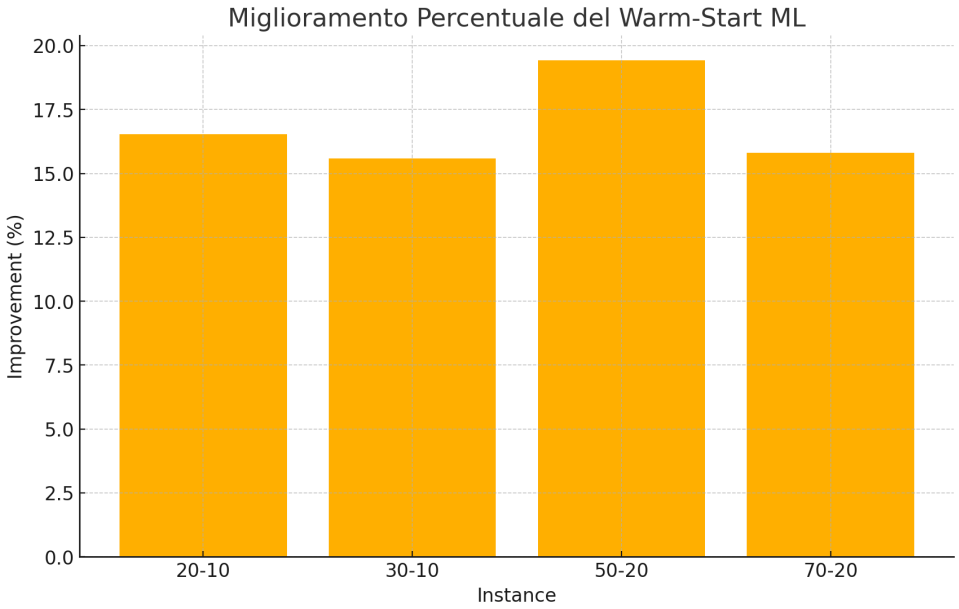


Figure 4.2: Percentage improvement of ML warm-start over the baseline.

Instance	ML Gap **without** LS (%)	ML Gap **with** LS (%)	LS Advantage (p.p.)
20–10	6.6	5.5	1.1
30–10	8.7	7.6	1.1
50–20	3.8	2.9	0.9
70–20	12.3	11.2	1.1

Table 4.2: Effect of local search on the percentage gap (ML).

Local optimization reduces the gap by about one percentage point on average; sometimes the cost remains unchanged because the greedy solution is already at a local minimum. The additional runtime is negligible (< 50), therefore LS remains enabled by default.

4.5 Results Analysis

The ML warm-start reduces energy costs by between 15% and 19% with respect to the baseline scheduling. In particular:

- 20 jobs, 10 machines: from €6.23 to €5.20 (–16.6%);
- 30 jobs, 10 machines: from €9.69 to €8.18 (–15.5%);
- 50 jobs, 20 machines: from €16.94 to €13.65 (–19.4%);
- 70 jobs, 20 machines: from €22.59 to €19.02 (–15.8%).

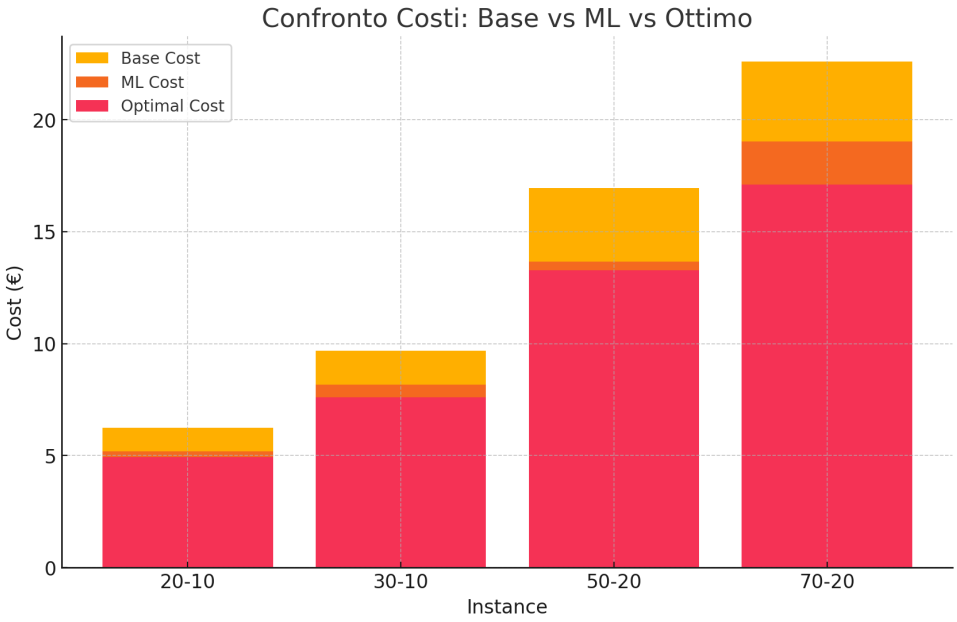


Figure 4.3: Power peak comparison: Greedy vs Optimum.

Initial gaps (26–32%) shrink to 3–11% with ML warm-start, saving over 20 percentage points.

4.6 Final Considerations

The tests confirm the effectiveness of the ML warm-start: near-optimal quality with very limited runtime. Local search provides a further (albeit modest) refinement with negligible computational impact, making the workflow suitable for day-to-day use in SMEs.

Bibliography

[1] Gurobi Optimization, LLC. (2024). *Gurobi Optimizer Reference Manual*. `https://www.gurobi.com`

[2] Scikit-learn Developers. (2024). *Machine Learning in Python*. `https://scikit-learn.org/`

[3] Wes McKinney. (2010). *Data Structures for Statistical Computing in Python*. In Proc. SciPy.